

Fluento Backend — 프로젝트 구조 및 모듈 설명

기술 스택

항목	내용
Language	Java 17 (Virtual Threads)
Framework	Spring Boot 4.0.0 / Spring Framework 7.0.1
Build	Gradle 9 (Groovy DSL)
DB	PostgreSQL 18 + JPA (Hibernate) + Flyway
Cache	Redis (Lettuce)
Auth	Spring Security + AWS Cognito (JWT / OAuth2 Resource Server)
AI	OpenAI Java SDK 2.4.0 (GPT-4o)
Streaming	Spring WebFlux Flux + SSE (Server-Sent Events)
Docs	SpringDoc OpenAPI (Swagger UI)

디렉터리 구조

fluento-backend/	
├─ build.gradle	# 의존성 및 빌드 설정
├─ CLAUDE.md	# 빌드·실행 명령어, 환경변수 정리
├─ PROJECT_OVERVIEW.md	# 이 파일
├─ 03_NEXT_STEPS_PROMPTS.md	# 다음 단계 프롬프트 모음
└─ src/main/	
├─ java/com/fluento/	
├─ FluentoBackendApplication.java	# 진입점
├─ config/	# 설정 클래스
├─ controller/	# REST API 엔드포인트
├─ domain/	# 엔티티 + JPA Repository
├─ dto/	# 요청/응답 DTO
├─ exception/	# 예외 클래스 + 전역 핸들러
└─ service/	# 비즈니스 로직
└─ resources/	
├─ application.yml	# 공통 설정
├─ application-dev.yml	# dev 프로파일 설정
└─ db/migration/	
└─ V1__Initial_schema.sql	# DB 스키마 (Flyway)

패키지별 역할

config/ — 애플리케이션 설정

파일	역할
SecurityConfig.java	JWT 검증, CORS, 인증 필터 체인 설정. <code>@Profile("!dev")</code> 로 dev 환경에서는 비활성화
DevSecurityConfig.java	<code>@Profile("dev")</code> - 가짜 JwtDecoder로 어떤 토큰이든 <code>sub="dev-user"</code> 로 처리. 개발용 인증 우회
DevDataInitializer.java	<code>@Profile("dev")</code> - 앱 시작 시 <code>googleId="dev-user"</code> 테스트 유저 자동 생성
JacksonConfig.java	ObjectMapper Bean 등록 (JavaTimeModule 포함). Spring Boot 4에서 자동 설정이 제거되어 명시 필요
RedisConfig.java	RedisTemplate, StringRedisTemplate Bean 설정. Jackson 직렬화 사용

controller/ — REST 엔드포인트

파일	엔드포인트	역할
AuthController	POST /api/v1/auth/register POST /api/v1/auth/logout	JWT에서 유저 정보 추출 후 회원 등록
UserController	GET/PATCH /api/v1/users/me	내 프로필 조회·수정
ChatRoomController	GET/POST/PATCH/DELETE /api/v1/chat/rooms	채팅방 목록·생성·수정·삭제·핀 고정
ChatMessageController	GET /api/v1/chat/rooms/{roomId}/messages POST /api/v1/chat/rooms/{roomId}/messages	메시지 목록 조회, 사용자 메시지 전송
ChatSSEController	GET /stream POST /initial-message GET /level	SSE AI 응답 스트리밍, 첫 인사말, 레벨 조회
HealthController	GET /api/v1/health	서버 상태 확인

domain/ — 데이터 모델

파일	DB 테이블	설명
User	users	소셜 로그인 유저. <code>google_id</code> , <code>email</code> , 앱 설정(언어·테마·알림) 포함
ChatRoom	chat_rooms	UUID PK. 유저 1명 + 캐릭터 1명당 1개 (유니크 제약). 현재 레벨, 메시지 수 관리
ChatMessage	chat_messages	UUID PK. 사용자/AI 메시지. 평가 결과(<code>isCorrect</code> , <code>score</code> , <code>feedbackJson</code>) 포함
LevelAssessment	level_assessments	메시지별 레벨 판단 기록. <code>confidence</code> , <code>indicators</code> (어휘·문법·유창성) 저장
Level (enum)	—	<code>BEGINNER</code> / <code>INTERMEDIATE</code> / <code>ADVANCED</code> . JSON에서 소문자로 직렬화 (<code>@JsonValue</code>)
SenderType (enum)	—	<code>USER</code> / <code>AI</code>
MessageType (enum)	—	<code>TEXT</code> / <code>IMAGE</code>

dto/ — 데이터 전송 객체

파일	역할
<code>ApiResponse<T></code>	공통 응답 래퍼 { <code>success</code> , <code>data</code> , <code>error</code> }. 모든 REST 응답에 사용
<code>ApiError</code>	에러 응답 포맷 { <code>code</code> , <code>message</code> , <code>statusCode</code> , <code>timestamp</code> }
<code>chat/SendMessageRequest</code>	사용자 메시지 전송 요청
<code>chat/ChatRoomResponse</code>	채팅방 정보 응답
<code>chat/ChatMessageResponse</code>	메시지 정보 응답
<code>chat/LevelAssessmentDTO</code>	레벨 판단 결과
<code>chat/LevelAdjustmentDTO</code>	레벨 조정 결과 (<code>shouldAdjust</code> , <code>from</code> , <code>to</code> , <code>reason</code>)
<code>chat/EvaluationDTO</code>	문장 평가 결과 (<code>isCorrect</code> , <code>score</code> , <code>feedback</code>)

exception/ — 예외 처리

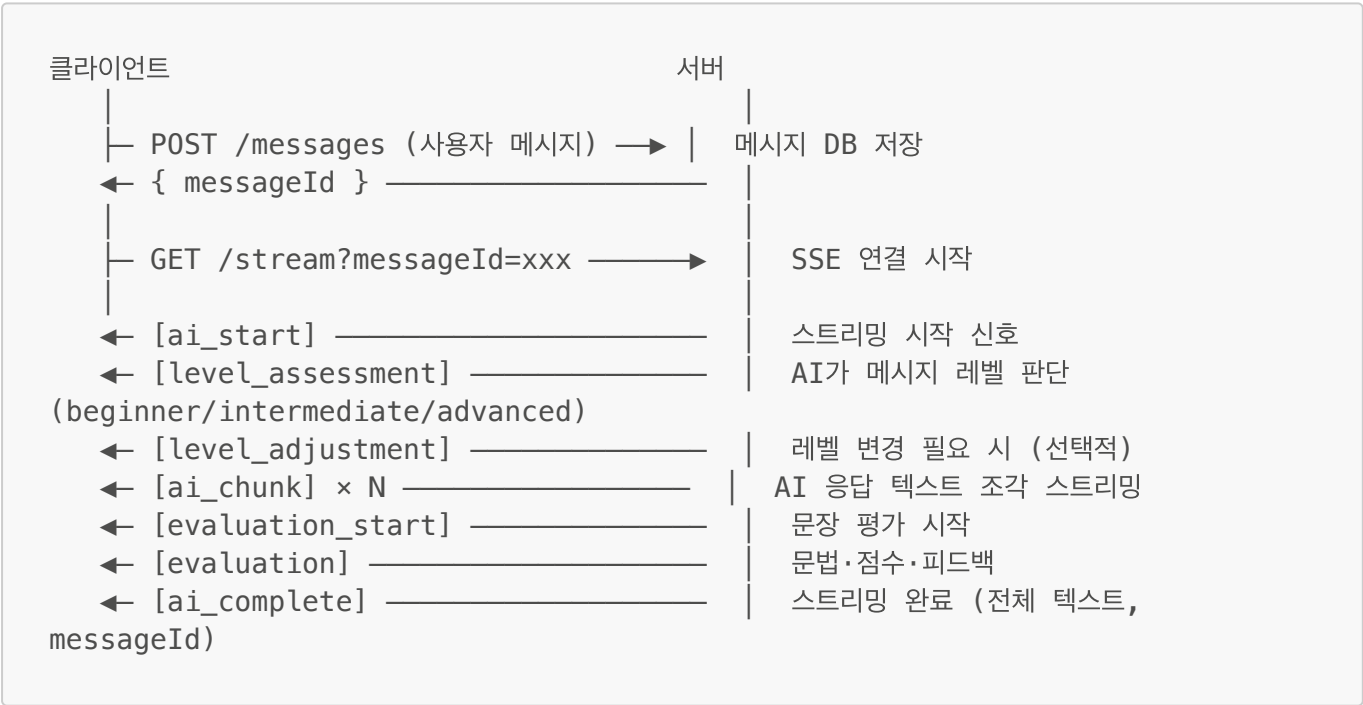
파일	역할
<code>GlobalExceptionHandler</code>	<code>@RestControllerAdvice</code> . 모든 예외를 <code>ApiResponse.fail()</code> 형태로 변환
<code>UserNotFoundException</code>	유저 없을 때 (404)
<code>ChatRoomNotFoundException</code>	채팅방 없을 때 (404)
<code>ChatRoomAlreadyExistsException</code>	동일 캐릭터 채팅방 중복 생성 시 (409)
<code>UnauthorizedException</code>	권한 없는 접근 시 (403)

service/ — 비즈니스 로직

파일	역할
UserService	유저 조회·생성·수정
ChatRoomService	채팅방 CRUD. 소유권 검증 포함
ChatMessageService	메시지 저장·조회
OpenAIService	GPT-4o API 호출. 스트리밍(Flux<String>)과 동기 호출 모두 지원. 레벨별 시스템 프롬프트 관리
AIResponseService	SSE 스트림 전체 흐름 조율 (아래 흐름 참조)
LevelAssessmentService	사용자 메시지를 AI로 분석해 레벨 판단. 결과를 DB 저장
LevelAdjustmentService	Redis에 최근 점수 기록. 연속 3회 ≥90점 → 레벨 업, 연속 2회 ≤50점 → 레벨 다운
EvaluationService	사용자 문장 문법·취향스 평가. 점수(0~100)와 피드백 반환

핵심 흐름: AI 응답 SSE 스트리밍

사용자가 메시지를 보내면 다음 순서로 SSE 이벤트가 전송됩니다.



인증 흐름

프론트엔드

→ Google 로그인 → AWS Cognito → JWT 발급

→ 모든 API 요청 헤더에 "Authorization: Bearer {JWT}" 첨부

- Spring Security가 Cognito 공개키로 JWT 검증
- jwt.getSubject() = google_id로 유저 식별

dev 프로필 우회:

```
# 앱 실행
./gradlew bootRun --args='--spring.profiles.active=dev'

# API 테스트 (아무 토큰이나 가능)
curl -H "Authorization: Bearer dev-token"
http://localhost:8080/api/v1/users/me
```

데이터베이스 스키마 요약

```
users (1)
├─ chat_rooms (N)          ← user_id + character_id UNIQUE
│   └─ chat_messages (N) ← room_id
│       └─ level_assessments (N) ← room_id, message_id
```

Redis 사용 목적

Key 패턴	저장 내용	용도
fluento:scores:{roomId}	최근 5개 점수 리스트	레벨 조정 판단 (연속 고득점/저득점 감지)

환경변수

변수	기본값	설명
DB_HOST	localhost	PostgreSQL 호스트
DB_USER	jeongsua	DB 유저명
DB_PASSWORD	1234	DB 비밀번호
OPENAI_API_KEY	(없음)	OpenAI API 키 (필수)
COGNITO_POOL_ID	—	AWS Cognito User Pool ID
AWS_REGION	—	AWS 리전